ESC112 Tutorial 2

SECTION: INTRODUCTION

- Pointers are special kinds of variables (or data types) that hold the **memory location of another variable** rather than storing a value directly.
- The "another variable" mentioned above can be of any type: `int`, `unsigned char`, `long double`, etc. Corresponding pointer declarations would be:

```
1 int *x;
2 unsigned char *y;
3 long double *z;
```

- In a computer, every data item occupies one or more contiguous memory cells called a **byte**. Typical sizes are:

Type	Typical Size
char	1 byte
int	4 bytes
float	4 bytes
double	8 bytes

  Memory cells are arranged sequentially and are identified by continuous **hexadecimal addresses**.

- When we declare

```
1 double d;
```

  the compiler automatically allocates 8 bytes of storage for `d`. The variable can also be referred to by its starting memory address.

  The address of a variable can be obtained using the unary `&` (address-of) operator.

```
1 double d, *pd = &d;
```

  Do not confuse `&` with the bitwise AND operator. The compiler distinguishes them automatically.

  (Whitespace after `*` or `&` is ignored. Typically no whitespace is used.)

- The pointer `pd` now holds the address of the first byte of `d`. The expression `*pd` behaves exactly like the variable `d`.

  Here `*` is called the **unary indirection operator**.

  Do not confuse this with the binary multiplication operator.

- Example:

```
1 double d, *pd = &d, e = *pd;
```

  Here `d` and `e` will contain the same value.

- Addresses are hexadecimal numbers. Use `%p`, `%x`, or `%X` to print pointer values.

```
1 #include <stdio.h>
2
3 int main(){
4     int u = 3, *pu = &u, v = *pu, *pv = &v;
5
6     printf("u = %d, &u = %p, pu = %p, *pu = %d\n", u, &u, pu, *pu);
7     printf("v = %d, &v = %x, pv = %X, *pv = %d\n", v, &v, pv, *pv);
8 }
```

Output

```
1 u = 3, &u = 0x7ffecd0c2c60, pu = 0x7ffecd0c2c60, *pu = 3
2 v = 3, &v = cd0c2c64, pv = CD0C2C64, *pv = 3
```

- The `&` operator can only be applied to ordinary variables (such as `x`) or array elements (`a[i]`). It cannot be applied to expressions such as `&(a+b)`.

- The `*` operator can only be applied to pointers. If `*p = u`, then `*p` can be used anywhere the variable `u` can be used.

```
1 #include <stdio.h>
2
3 int main(){
4     int u1, u2, v = 3, *pv = &v;
5
6     u1 = 2 * (v + 5);
7     u2 = 2 * (*pv + 5);
8
9     printf("u1 = %d, u2 = %d", u1, u2);
10 }
```

Output: `16 16`

- The expression `*p` can also appear on the left-hand side of an assignment.

```c
#include <stdio.h>

int main(){
    int v = 3, *pv = &v;

    printf("*pv = %d, v = %d\n", *pv, v);

    *pv = 0;

    printf("*pv = %d, v = %d\n", *pv, v);
}
```

Output

```
*pv = 3, v = 3
*pv = 0, v = 0
```

- Pointer variables can point to numeric variables, character variables, arrays, functions, or other pointer variables.
- A pointer variable can also be assigned the value of another pointer of the same type.

```c
int x, *px = &x, *pv = px;
```

- Ordinary variables cannot be assigned arbitrary addresses. Therefore `&x` cannot appear on the left-hand side of an assignment.
- Any pointer can be assigned a special integer value called `NULL` (defined as 0). This indicates that the pointer does not refer to any valid address.

```c
int *x = NULL;
```

- Unary operators have relatively high precedence and associate from right to left.

SECTION: PASSING POINTERS TO FUNCTIONS

Functions can take pointer arguments. The function can access the memory location and modify the value stored there. This is an extremely useful way of implicitly returning multiple values from a function.

```c
#include <stdio.h>

void func1(int u, int v);
void func2(int *, int *);

int main(){
    int u = 1, v = 3;

    printf("u = %d, u = %d\n", u, v);

    func1(u, v);
    printf("u = %d, u = %d\n", u, v);

    func2(&u, &v);
    printf("u = %d, u = %d\n", u, v);
}

void func1(int u, int v){
    u = v = 0;
}

void func2(int *pu, int *pv){
    *pu = *pv = 0;
}
```

Output

```
u = 1, u = 3
u = 1, u = 3
u = 0, u = 0
```

Arrays and Function Arguments

An array name actually represents the address of its first element. Therefore, an array behaves like a pointer when passed to a function.

For this reason, the & operator should **not** be used when passing an array name as an argument.

Within a function definition, an array parameter can be declared either as

```c
char line[];
```

or

```c
char *line;
```

Example Program

Problem: Count the number of vowels, consonants, digits, whitespace characters and other characters in a line of text.

this uses the readline function wrote earlier that reads a line from standard input.

```c
/* Count characters in a line of text */

#include <stdio.h>
#include <ctype.h>

void readline(char line[]){
    int c, i = 0;;
    while((c = getchar()) != '\n')
        line[i++] = c;
    line[i] = '\0'
}

void scan_line(char line[], int *pv, int *pc, int *pd,
               int *pw, int *po);

int main()
{
    char line[80];
    int vowels = 0;
    int consonants = 0;
    int digits = 0;
    int whitespc = 0;
    int other = 0;

    readline(line);

    scan_line(line, &vowels, &consonants,
              &digits, &whitespc, &other);

    printf("\nNo. of vowels: %d", vowels);
    printf("\nNo. of consonants: %d", consonants);
    printf("\nNo. of digits: %d", digits);
    printf("\nNo. of whitespace characters: %d", whitespc);
    printf("\nNo. of other characters: %d\n", other);

    return 0;
}

void scan_line(char line[], int *pv, int *pc, int *pd,
               int *pw, int *po)
{
    char c;
    int count = 0;

    while ((c = toupper(line[count])) != '\0')
    {
        if (c=='A'||c=='E'||c=='I'||c=='O'||c=='U')
            ++(*pv);
        else if (c >= 'A' && c <= 'Z')
            ++(*pc);
        else if (c >= '0' && c <= '9')
            ++(*pd);
        else if (c == ' ' || c == '\t')
            ++(*pw);
        else
            ++(*po);

        ++count;
    }
}
```

Why `&` is Needed in `scanf`

This also explains why `scanf` requires the `&` operator for ordinary variables.

```
#include <stdio.h>

int main(){
    char str[20];
    int a, b[20];

    scanf("%s %d %d", str, &a, &b[5]);
}
```

Functions Returning Pointers

Functions may also return pointer values.

```
int *func(){
    int *x;

    ...
    x = ...;
    ...

    return x;
}
```

SECTION: POINTERS AND 1-D ARRAYS

Pointers and One-Dimensional Arrays

Recall that an array name is actually a pointer to the first element of the array. Therefore, if `x` is a one-dimensional array, then the address of the first array element can be written in two equivalent ways:

```
&x[0]
x
```

Similarly, the address of the second array element can be written as

```
&x[1]
x + 1
```

In general, the address of the `i`-th element can be written as

```
&x[i]
x + i
```

Thus there are two equivalent ways to represent the address of any array element:

- Using the array element preceded by `&`
- Adding the index to the array name

Understanding the Expression `x + i`

The expression `(x + i)` represents a special type of expression.

- `x` represents the address of the first element of the array.
- `i` is an integer offset.
- The array elements may be of type `char`, `int`, `float`, etc.

Therefore we are not simply adding numbers. Instead we are specifying an address that is `i` array elements beyond the first element.

Hence the expression `(x + i)` is best understood as a symbolic address specification rather than a simple arithmetic expression.

The value `i` is often called the **offset**.

Automatic Scaling by the Compiler

Different data types occupy different numbers of memory cells depending on the computer architecture.

For example:

Type	Typical Size
char	1 byte
int	2 or 4 bytes
float	4 bytes
double	8 bytes

When writing `x + i`, the programmer does not need to worry about these sizes. The C compiler automatically scales the address according to the size of the array element.

Thus you only specify:

- the base address (`x`)
- the offset (`i`)

Relationship Between Array Notation and Pointer Notation

Since

```
1 &x[i]  ==  x + i
```

it follows that

```
1 x[i]   ==  *(x + i)
```

Thus the two expressions are interchangeable.

Expression	Meaning
x[i]	value of the i-th element
*(x+i)	value of the i-th element
&x[i]	address of the i-th element
x+i	address of the i-th element

Example Program

The following program illustrates the relationship between array elements and their addresses.

```
#include <stdio.h>

int main()
{
    int x[10] = {10,11,12,13,14,15,16,17,18,19};
    int i;

    for(i = 0; i <= 9; ++i){

        /* display element value */
        printf("\ni = %d  x[i] = %d  *(x+i) = %d",
                i, x[i], *(x+i));

        /* display element address */
        printf("   &x[i] = %X  x+i = %X",
                &x[i], x+i);
    }
}
```

Typical output:

```
i=0 x[i]=10 *(x+i)=10 &x[i]=72 x+i=72
i=1 x[i]=11 *(x+i)=11 &x[i]=74 x+i=74
...
i=9 x[i]=19 *(x+i)=19 &x[i]=84 x+i=84
```

This output shows that

- `x[i]` and `*(x+i)` represent the same value
- `&x[i]` and `x+i` represent the same address

Assignment Using Pointer Notation

When assigning values to array elements, the left-hand side may be written either using array notation or pointer notation.

```
x[i] = 10;
*(x+i) = 10;
```

Both statements assign the same value to the `i`-th array element.
However, addresses cannot be assigned to array elements.

The following statement is illegal:

```
&x[2] = &x[1];    // illegal
```

If we wish to store an address, we must use a pointer variable.

```
int *p;
p = &x[1];
p = x + 1;
```

Example

```
#include <stdio.h>

int main(){

    int line[80];
    int *pl;

    /* assign values */

    line[2] = line[1];
    line[2] = *(line + 1);
    *(line + 2) = line[1];
    *(line + 2) = *(line + 1);

    /* assign addresses */

    pl = &line[1];
    pl = line + 1;
}
```

All four value assignments copy the value of `line[1]` into `line[2]`.
The last two statements assign the address of `line[1]` to the pointer `pl`.

Strings and Character Pointers

A string may be represented either as a character array or as a character pointer.
Example using arrays:

```c
#include <stdio.h>

char x[] = "This string is declared externally\n\n";

int main()
{
    char y[] = "This string is declared within main";

    printf("%s", x);
    printf("%s", y);
}
```

The same program using pointer variables:

```c
#include <stdio.h>

char *x = "This string is declared externally\n\n";

int main()
{
    char *y = "This string is declared within main";

    printf("%s", x);
    printf("%s", y);
}
```

In this version, the pointer variables x and y point to the beginning of their respective strings. Execution of either program produces

```
This string is declared externally

This string is declared within main
```